

Instructions

Project Assignment: Robot Model Telemetry Refactor

This assignment is for an advanced student who will plan and execute a full refactor of telemetry handling in the Robot Model.

The current design pushes telemetry through the hierarchy as a method parameter and also lets several concrete classes write directly to telemetry on their own. That split ownership has already created inconsistent behavior, dead telemetry hooks, and autonomous code that bypasses the object model.

Your job is to replace that pattern with a single, consistent telemetry architecture.

Assignment Goal

Refactor the Robot Model so that the single telemetry instance created by the OpMode is injected into each abstract layer of the model and owned there, instead of being passed down every loop through *updateTelemetry(Telemetry telemetry)* methods.

The target architecture must also give the autonomous-facing layers their own telemetry write methods so autonomous code can report state without taking raw Telemetry parameters everywhere.

This assignment includes both design work and implementation work.

Core Requirements

Your refactor must satisfy all of the following:

1. The OpMode remains the source of the one telemetry instance used for the run.
2. Telemetry is injected into the Robot Model at construction time, not threaded through the loop as a method argument.
3. Each abstract manual layer has a telemetry-writing responsibility that uses its injected telemetry object.
4. Each abstract autonomous layer also has a telemetry-writing responsibility.
5. Telemetry ownership is consistent across the full hierarchy, not handled one way in Billy and another way elsewhere.
6. The final design defines one clear rule for who is allowed to call *telemetry.update()*.
7. The refactor fixes the current telemetry bugs and failure modes rather than merely moving them around.

Architectural Intent

The desired end state is:

OpMode creates the robot and owns FTC lifecycle.

Robot, DriveTrain, MechAssembly, and component-level abstractions each receive telemetry as part of construction or controlled initialization. Those layers write telemetry through their own telemetry reference rather than accepting Telemetry as a loop-time parameter.

The autonomous-facing abstractions also expose telemetry-writing entry points.

Autonomous strategies should be able to report status through the robot model instead of requiring raw telemetry to be passed directly into each strategy.

Scope

At minimum, this assignment applies to the following architectural surfaces:

- Robot
- DriveTrain
- MechAssembly
- MechComponent
- Robot.AutonomousRobot
- DriveTrain.AutonomousDriving
- MechAssembly.AutonomousMechBehaviors
- Any autonomous component surface that needs to participate in the new telemetry model

Concrete implementations in current use must also be updated so the architecture is actually exercised:

- BillyRobot
- Wildbots2025
- MecanumDrive
- StandardTankDrive
- OneStickTank
- BillyMA
- CascadeArm
- ExampleIntakeAssembly
- Relevant mech components and autonomous helpers

Required Deliverables

You must submit all of the following:

1. A design write-up created before major implementation begins.
2. A bug inventory describing the problems caused by the current telemetry design.
3. The completed refactor.
4. A verification report describing what you tested and what behavior you confirmed.
5. A short reflection explaining tradeoffs, compromises, and any follow-up work still needed.

Required Planning Write-Up

Your design write-up must include these sections:

1. Current-State Map

Describe how telemetry currently flows from OpMode into the Robot Model and where telemetry is written today.

1. Ownership Rule

State exactly which layer owns the telemetry reference at each point in the hierarchy.

1. Update Rule

State exactly where *telemetry.update()* is allowed to happen and why.

1. Manual-Layer Design

Explain how the manual layers will write telemetry after Telemetry is no longer passed as a loop-time argument.

1. Autonomous-Layer Design

Explain how AutonomousRobot, AutonomousDriving, AutonomousMechBehaviors, and any needed autonomous component surfaces will write telemetry.

1. Migration Plan

List the implementation order you will follow and explain why that order reduces breakage.

1. Risk Analysis

Identify the main refactor risks, including constructor churn, partially migrated classes, and behavior regressions.

1. Verification Plan

Describe how you will prove the new design works in both TeleOp and Autonomous.

The write-up must describe architecture and rationale, not just a list of files to edit.

Required Bug Analysis

You must identify the bugs and failure modes caused by the current implementation. Your analysis must include file references, the observed symptom, and the architectural root cause.

At minimum, your write-up must address the following current problems:

1. Duplicate telemetry flushes in the TeleOp loop.

Evidence to inspect:

Robot

TeleOp

BillyTeleOpBLUE

BillyTeleOpRED

1. Mid-cycle telemetry writes and updates from lower layers, which fragment the telemetry lifecycle.

Evidence to inspect:

BillyMA

BillyRapidFire

LimelightAutoTarget

1. Telemetry hooks that exist in the API but do not actually report data.

Evidence to inspect:

DoubleShooter

SpinnyIntake

StandardTankDrive

OneStickTank

1. Telemetry strategy objects that are configured but ignored by the implementation.

Evidence to inspect:

PusherServo

BillyMA

1. Assemblies that fail to report the telemetry of all relevant child components.

Evidence to inspect:

BillyMA

ExampleIntakeAssembly

1. Autonomous code that bypasses the Robot Model and accepts raw telemetry because the autonomous abstractions do not provide their own telemetry path.

Evidence to inspect:

Robot

BillyRobot

BillyRapidFire

LimelightAutoTarget

ATagL1Strategy

ATagL2Strategy

You may identify additional bugs beyond these six. You should.

Constraints

Your refactor must follow these constraints:

- Do not solve this by keeping the same pass-through design and only renaming methods.
- Do not make OpModes thicker.
- Do not introduce a second telemetry object.
- Do not require autonomous strategies to keep accepting raw Telemetry just because the base layers are inconvenient.
- Do not silently remove useful telemetry in order to simplify the refactor.
- Do not bury *telemetry.update()* calls all over the hierarchy.

Recommended Execution Steps

Follow this order unless your design write-up justifies a better sequence:

1. Inventory the current telemetry path.

Produce a dependency map showing where telemetry is stored, passed, and flushed today.

1. Define the final telemetry contract.

Decide what each abstract layer is allowed to do with telemetry and what method each layer exposes for writing it.

1. Refactor the abstract manual-layer contracts first.

Change the base model so the architecture itself reflects the new telemetry ownership rule.

1. Refactor the abstract autonomous-layer contracts next.

Add autonomous telemetry-writing responsibilities before touching every concrete strategy.

1. Migrate concrete drive trains, assemblies, and components.

Update implementations so they use owned telemetry rather than loop-time parameters.

1. Fix the known dead and missing telemetry paths.

Do not leave behind components that technically compile but still fail to report.

1. Migrate robot-level helpers and autonomous strategies.

Remove raw telemetry dependencies where the new autonomous surfaces should now cover them.

1. Simplify the OpModes.

The final OpModes should construct the robot, run the loop, and follow the one approved flush rule.

1. Verify both TeleOp and Autonomous behavior.

Confirm that telemetry still appears where expected and that it now follows one consistent lifecycle.

1. Document what changed.

Record which bugs were fixed, what design decisions were made, and what limitations remain.

Acceptance Criteria

The project is not complete until all of the following are true:

- The Robot Model no longer depends on passing Telemetry downward every loop as the standard telemetry mechanism.
- The abstract manual layers have a consistent telemetry ownership model.
- The abstract autonomous layers have telemetry-writing methods.
- Concrete robots and subsystems actually use the new design.
- There is one clearly documented rule for when *telemetry.update()* happens.
- Telemetry bugs identified in the bug inventory are fixed or explicitly justified as deferred work.
- TeleOp remains thin.
- Autonomous code can report status without scattering raw Telemetry parameters through strategy constructors and helper functions.
- The student can explain why the new design is architecturally better, not just say that it compiles.

Verification Expectations

Your verification report must include:

1. A manual test checklist for at least one TeleOp robot and one Autonomous flow.
2. Evidence that telemetry appears from the drivetrain, mech assembly, and at least one lower-level mechanism.
3. Evidence that autonomous telemetry is visible through the new architecture.
4. Evidence that the final design does not rely on multiple competing telemetry update points in the same cycle.
5. A list of any telemetry that still does not exist, and whether that is intentional.

Evaluation Criteria

You will be evaluated on:

- Accuracy of your current-state analysis
- Quality of your bug identification
- Quality of your architectural reasoning
- Discipline of your migration plan
- Completeness of the implementation
- Quality of your verification
- Ability to explain tradeoffs clearly

Final Reflection Prompt

At the end of the project, write a short reflection answering these questions:

1. What was the most important design mistake in the original telemetry architecture?
2. What changed at the abstract-layer level versus the concrete-class level?
3. What telemetry bug was the easiest to miss during code review?
4. What part of the refactor was structurally necessary but not obvious at first?
5. What follow-up refactor would you recommend next?

Definition of Success

Success means the student can defend a coherent answer to this question:

"Why is telemetry now a first-class responsibility of the Robot Model and its autonomous surfaces, instead of a loop-time argument that leaks through the hierarchy?"

If that answer is not clear, the refactor is not finished.

Deliverables

1. A design write-up created before major implementation begins.
2. A bug inventory describing the problems caused by the current telemetry design.
3. The completed refactor.
4. A verification report describing what you tested and what behavior you confirmed.
5. A short reflection explaining tradeoffs, compromises, and any follow-up work still needed.

Plan

Robot Model Telemetry Refactor

Plan Steps

1. Inventory the current telemetry path.

Produce a dependency map showing where telemetry is stored, passed, and flushed today.

2. Define the final telemetry contract.

Decide what each abstract layer is allowed to do with telemetry and what method each layer exposes for writing it.

3. Refactor the abstract manual-layer contracts first.

Change the base model so the architecture itself reflects the new telemetry ownership rule.

4. Refactor the abstract autonomous-layer contracts next.

Add autonomous telemetry-writing responsibilities before touching every concrete strategy.

5. Migrate concrete drive trains, assemblies, and components.

Update implementations so they use owned telemetry rather than loop-time parameters.

6. Fix the known dead and missing telemetry paths.

Do not leave behind components that technically compile but still fail to report.

7. Migrate robot-level helpers and autonomous strategies.

Remove raw telemetry dependencies where the new autonomous surfaces should now cover them.

8. Simplify the OpModes.

The final OpModes should construct the robot, run the loop, and follow the one approved flush rule.

9. Verify both TeleOp and Autonomous behavior.

Confirm that telemetry still appears where expected and that it now follows one consistent lifecycle.

10. Document what changed.

Record which bugs were fixed, what design decisions were made, and what limitations remain.

Current Telemetry Flow

Prompt: “Can you: Produce a dependency map showing where telemetry is stored, passed, and flushed today and decide what each abstract layer is allowed to do with telemetry and what method each layer exposes for writing it.”

Where Telemetry is Stored

- **OpModes:** Own the single `Telemetry` instance from FTC framework
- **BillyMA:** Stores telemetry reference as `private final Telemetry telemetry`
- **BillyRapidFire:** Stores telemetry reference as `private final Telemetry telemetry`
- **LimelightAutoTarget:** Stores telemetry reference as `private final Telemetry telemetry`

Where Telemetry is Passed

`OpMode.telemetry` ↓ (constructor) → `BillyRobot(hardwareMap, telemetry, tagID)` ↓ (constructor) → `BillyMA(hardwareMap, telemetry)` ↓ (constructor) → `BillyRapidFire(auton, 3, telemetry)` ↓ (constructor) → `LimelightAutoTarget(limelight, turret, telemetry, tagID)`

`OpMode.telemetry` ↓ (every loop) → `Robot.updateTelemetry(telemetry)` ↓ (delegates) → `DriveTrain.updateTelemetry(telemetry)` → `MechAssembly.updateTelemetry(telemetry)`

`OpMode.telemetry` ↓ (strategy creation) → `ATagL1Strategy.track(robot, telemetry, opMode, tagID)` → `ATagL2Strategy.track(robot, telemetry, opMode, tagID)` → `BasicApriltagTracking.trackTag(robot, telemetry, opMode)`

Where Telemetry is Flushed (`telemetry.update()`)

Multiple Competing Update Points:

1. **OpModes** (proper location):
 - `BillyTeleOpBLUE`: `telemetry.update()` in main loop
 - `TeleOp`: `telemetry.update()` in main loop
2. **Robot Layer** (duplicate):
 - `Robot.updateTelemetry()`: calls `telemetry.update()` at end
3. **State Machines** (mid-cycle fragmentation):
 - `BillyRapidFire`: calls `telemetry.update()` in `startShooter()`, `fire()`, `retract()`, `stopShooter()`

- `LimelightAutoTarget`: calls `telemetry.update()` in `rotateCCW()`, `rotateCW()`, `stopTurret()`

4. **Autonomous Strategies** (mid-cycle fragmentation):

- `ATagL1Strategy.lookForTag()`: multiple `telemetry.update()` calls
- `ATagL2Strategy.lookForTag()`: multiple `telemetry.update()` calls
- `BasicApriltagTracking.lookForTag()`: no update calls (missing)

Dead/Missing Telemetry Hooks

- `DoubleShooter.update()`: Empty implementation
- `SpinnyIntake.update()`: Empty implementation
- `StandardTankDrive.updateTelemetry()`: Empty implementation
- `OneStickTank.updateTelemetry()`: Empty implementation
- `ExampleIntakeAssembly.updateTelemetry()`: Only reports claw, ignores intake

New Telemetry Ownership

New Telemetry Contract Design

Ownership Rules

Layer	Telemetry Ownership	Responsibility
OpMode	Creates and owns the single instance	Calls robot telemetry methods, controls final flush
Robot	Receives at construction, owns reference	Coordinates telemetry from subsystems
DriveTrain	Receives at construction, owns reference	Reports drive-specific data
MechAssembly	Receives at construction, owns reference	Coordinates component telemetry
MechComponent	Receives at construction, owns reference	Reports component-specific data
Autonomous Layers	Access through owned telemetry reference	Report autonomous state and actions

Update Rule

Single Point of Control: Only the **OpMode** is allowed to call `telemetry.update()`. All other layers write data but never flush.

Current Bug Inventory

Prompt: “can you make a bug inventory describing the problems caused by the current telemetry design.”

Bug #1: Duplicate Telemetry Flushes in TeleOp Loop

Files: `Robot.java`, `BillyTeleOpBLUE.java`, `BillyTeleOpRED.java`, `TeleOp.java`

Symptom: Multiple `telemetry.update()` calls in the same loop cycle, causing unnecessary overhead and potential display issues.

Evidence:

```
// BillyTeleOpBLUE.java - Line 20-25
while (opModeIsActive()) {
    robot.update(gamepad1, gamepad2);
    robot.updateTelemetry(telemetry); // Calls telemetry.update()
internally
    telemetry.update();               // DUPLICATE: Second flush in same
cycle
}

// Robot.java - Line 35-39
public void updateTelemetry(Telemetry telemetry) {
    driveTrain.updateTelemetry(telemetry);
    mechAssembly.updateTelemetry(telemetry);
    telemetry.update(); // First flush happens here
}
```

Root Cause: The abstract `Robot` class assumes it owns the telemetry flush responsibility, but OpModes also call `telemetry.update()`, creating competing ownership.

Bug #2: Mid-Cycle Telemetry Writes and Updates

Files: `BillyMA.java`, `BillyRapidFire.java`, `LimelightAutoTarget.java`

Symptom: Telemetry data is written and flushed multiple times within a single execution cycle, fragmenting the telemetry lifecycle and potentially overwriting data.

Evidence:

```
// BillyRapidFire.java - Lines 35-45
public IState startShooter() {
    telemetry.addLine("startShooter");
    telemetry.update(); // Mid-cycle flush #1
    // ... more code
}

public IState fire() {
    telemetry.addLine("fire");
    telemetry.update(); // Mid-cycle flush #2
    // ... more code
}

// LimelightAutoTarget.java - Lines 75-85
public IState rotateCCW(double tx){
    // ... logic
    telemetry.addData("Turret CCW: ", power);
    telemetry.update(); // Mid-cycle flush during state execution
    return lookForTag();
}
```

Root Cause: State machines and autonomous helpers treat telemetry as immediately flushable rather than as data to be collected and flushed once per cycle.

Bug #3: Dead Telemetry Hooks

Files: `DoubleShooter.java`, `SpinnyIntake.java`, `StandardTankDrive.java`, `OneStickTank.java`

Symptom: Components have telemetry update methods that exist in the API but provide no data, creating false expectations.

Evidence:

```
// DoubleShooter.java - Lines 65-67
@Override
public void update(Telemetry telemetry) {
    // EMPTY: No telemetry data reported despite having telemetryStrategy
}

// SpinnyIntake.java - Lines 85-87
@Override
void update(Telemetry telemetry) {
    // EMPTY: No telemetry data reported
}

// StandardTankDrive.java - Lines 75-79
@Override
public void updateTelemetry(Telemetry telemetry) {
    // EMPTY: Tank drive provides no feedback
}

// OneStickTank.java - Lines 95-98
@Override
public void updateTelemetry(Telemetry telemetry) {
    // EMPTY: No telemetry despite complex Leroy Jenkins state
}
```

Root Cause: The abstract contracts require telemetry methods, but implementations don't fulfill the contract meaningfully.

Bug #4: Ignored Telemetry Strategy Objects

Files: `PusherServo.java`, `BillyMA.java`

Symptom: Components are configured with telemetry strategy objects, but the strategies are either ignored or inconsistently used.

Evidence:

```
// PusherServo.java - Constructor accepts telemetryStrategy
public PusherServo(HardwareMap hardwareMap,
                   String servoNameR,
                   BallPusherControlStrategy strategy,
                   PusherTelemetryStrategy telemetryStrategy) {
    // telemetryStrategy is stored but...
}

// PusherServo.java - update() ignores the strategy
@Override
public void update(Telemetry telemetry) {
    telemetry.addData("Right Pusher Position", "%.2f",
servoR.getPosition());
    // IGNORED: telemetryStrategy.update() is never called
}

// BillyMA.java - flywheel has telemetry strategy but DoubleShooter
ignores it
flywheel = new DoubleShooter(hardwareMap, "flywheelMotorF",
"flywheelMotorB",
    // ... control strategy
    ((motorF, motorB, telemetry) -> {
        telemetry.addData("power:", motorF.getPower()); // Strategy
defined
    }));
// But DoubleShooter.update() is empty, so strategy never executes
```

Root Cause: Inconsistent application of the strategy pattern - some components use strategies, others hardcode telemetry, and some ignore their strategies entirely.

Bug #5: Missing Component Telemetry in Assemblies

Files: BillyMA.java, ExampleIntakeAssembly.java

Symptom: Assemblies fail to report telemetry from all their child components, creating incomplete system visibility.

Evidence:

```
// BillyMA.java - Lines 110-114
@Override
public void updateTelemetry(Telemetry telemetry) {
    ballPusher.update(telemetry); // Reports pusher
    flywheel.update(telemetry);    // Reports flywheel (but it's empty)
    turret.update(telemetry);      // Reports turret
    // MISSING: intake.update() never called - no intake telemetry
}

// ExampleIntakeAssembly.java - Lines 55-57
@Override
public void updateTelemetry(Telemetry telemetry) {
    claw.update(telemetry); // Reports claw only
    // MISSING: intake.update() never called
}
```

Root Cause: Manual coordination of component telemetry in assemblies is error-prone and incomplete.

Bug #6: Raw Telemetry Dependencies in Autonomous Code

Files: Robot.java, BillyRobot.java, BillyRapidFire.java, LimelightAutoTarget.java, ATagL1Strategy.java, ATagL2Strategy.java

Symptom: Autonomous strategies and helpers require raw `Telemetry` parameters because the autonomous abstractions provide no telemetry path, bypassing the robot model.

Evidence:

```
// ATagL1Strategy.java - Raw telemetry passed everywhere
public static IAutonStrategy track(BillyRobot robot, Telemetry telemetry,
...) {
    // Raw telemetry passed to strategy
}

public static IState lookForTag(BillyRobot robot, Telemetry telemetry, int
tagId) {
```

```

    // Raw telemetry used directly
    telemetry.addLine("looking for tag");
    telemetry.update();
}

// BillyRapidFire.java - Constructor requires raw telemetry
public BillyRapidFire(BillyMA.AutonomousBillyMA ma, int ballCount,
Telemetry tm) {
    telemetry = tm; // Raw telemetry stored
}

// Robot.AutonomousRobot has no telemetry methods
public abstract class AutonomousRobot {
    // NO telemetry support - forces strategies to use raw telemetry
}

```

Root Cause: The autonomous-facing abstractions (`Robot.AutonomousRobot`, `AutonomousDriving`, `AutonomousMechBehaviors`) provide no telemetry-writing methods, forcing autonomous code to bypass the object model and accept raw telemetry references.

Bug #7: Inconsistent Telemetry Lifecycle Management

Files: `ATagL2Strategy.java`, `BillyMA.java`

Symptom: Some autonomous code calls `telemetry.clear()` mid-execution, potentially removing data written by other parts of the system.

Evidence:

```

// ATagL2Strategy.java - Line 155
public static IState IntakeBalls(BillyRobot robot, Telemetry telemetry) {
    return () -> {
        telemetry.clear(); // DESTRUCTIVE: Clears all telemetry data
        telemetry.addLine("intaking balls");
        // ... rest of method
    };
}

// BillyMA.java - giveInstructions() writes telemetry immediately

```

```
if(gamepad.a && BRF.isComplete()) {  
    BRF.reset(3);  
    telemetry.addLine("Start Rapid Fire");  
    telemetry.update(); // Immediate flush during instruction processing  
}
```

Root Cause: No clear ownership of telemetry lifecycle - different parts of the system assume they can clear, write, and flush telemetry independently.

Summary of Architectural Problems

1. **Split Ownership:** Multiple layers assume they can call `telemetry.update()`
2. **Fragmented Lifecycle:** Telemetry written and flushed at arbitrary points in execution
3. **Incomplete Implementation:** Required telemetry methods exist but provide no data
4. **Inconsistent Patterns:** Some components use strategies, others don't, some ignore their strategies
5. **Missing Coordination:** Assemblies don't comprehensively report child component state
6. **Architectural Bypass:** Autonomous code forced to use raw telemetry because proper abstractions don't exist
7. **Lifecycle Conflicts:** Different parts of the system compete for control of telemetry clearing and flushing

These problems stem from the fundamental architectural flaw of **passing telemetry as a method parameter** rather than **injecting it as an owned responsibility** at each layer.

REFACTORS

Many many subtabs...

Robot.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Robots;

import com.qualcomm.robotcore.hardware.Gamepad;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.DriveTrain;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies.MechAssembly;
import org.firstinspires.ftc.vision.VisionPortal;

public abstract class Robot
{
    public abstract class AutonomousRobot
    {
        protected final Telemetry telemetry;

        public AutonomousRobot(
            DriveTrain.AutonomousDriving driveTrain,
            MechAssembly.AutonomousMechBehaviors mechAssembly,
            Telemetry telemetry)
        {
            this.telemetry = telemetry;
        }

        public void reportStatus(String message) {
            telemetry.addLine(message);
        }

        public void reportData(String key, Object value) {
            telemetry.addData(key, value);
        }

        public void clearTelemetry() {
            telemetry.clear();
        }
    }
}

protected interface IControlStrategy { }

public abstract <T extends AutonomousRobot> T getAutonomousRobot();

protected final Telemetry telemetry;
protected VisionPortal visionPortal;
protected DriveTrain driveTrain;
protected MechAssembly mechAssembly;

```

```

protected Robot(Telemetry telemetry) {
    this.telemetry = telemetry;
}

protected void initializeSubsystems() {
    if (driveTrain != null) {
        driveTrain.initializeTelemetry(telemetry);
    }
    if (mechAssembly != null) {
        mechAssembly.initializeTelemetry(telemetry);
    }
}

public void updateTelemetry() {
    if (driveTrain != null) {
        driveTrain.updateTelemetry();
    }
    if (mechAssembly != null) {
        mechAssembly.updateTelemetry();
    }
    // ONLY Robot calls telemetry.update()
    telemetry.update();
}

public void update(Gamepad gamepad1, Gamepad gamepad2)
{
    driveTrain.drive(gamepad1);
    mechAssembly.giveInstructions(gamepad2);
}
}

```

DriveTrain.java

```

package org.firstinspires.ftc.teamcode.RobotModel.DriveTrain;

import com.qualcomm.robotcore.hardware.Gamepad;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public abstract class DriveTrain
{
    public abstract class AutonomousDriving {
        protected final Telemetry telemetry;

        protected AutonomousDriving(Telemetry telemetry) {
            this.telemetry = telemetry;
        }

        public void reportStatus(String message) {
            telemetry.addLine(message);
        }

        public void reportData(String key, Object value) {
            telemetry.addData(key, value);
        }
    }

    protected Telemetry telemetry;

    public abstract <T extends AutonomousDriving> T getAutonomousDriving();
    public abstract void drive(Gamepad gamepad);
    public abstract void initializeTelemetry(Telemetry telemetry);
    public abstract void updateTelemetry();
}

```

MechAssembly.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies;

import com.qualcomm.robotcore.hardware.Gamepad;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public abstract class MechAssembly
{
    public abstract class AutonomousMechBehaviors {
        protected final Telemetry telemetry;

        protected AutonomousMechBehaviors(Telemetry telemetry) {
            this.telemetry = telemetry;
        }

        public void reportStatus(String message) {
            telemetry.addLine(message);
        }

        public void reportData(String key, Object value) {
            telemetry.addData(key, value);
        }
    }

    protected Telemetry telemetry;

    public abstract <T extends AutonomousMechBehaviors> T getAutonomousBehaviors();
    public abstract void giveInstructions(Gamepad gamepad);
    public abstract void initializeTelemetry(Telemetry telemetry);
    public abstract void updateTelemetry();
}

```

MechComponent.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.Gamepad;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public abstract class MechComponent
{
    public abstract class AutonomousComponentBehaviors {
        protected final Telemetry telemetry;

        protected AutonomousComponentBehaviors(Telemetry telemetry) {
            this.telemetry = telemetry;
        }

        public void reportStatus(String message) {
            telemetry.addLine(message);
        }

        public void reportData(String key, Object value) {
            telemetry.addData(key, value);
        }
    }

    public abstract <T extends AutonomousComponentBehaviors> T getAutonomousBehaviors();
    protected interface IControlStrategy { }

    protected IControlStrategy strategy;
    protected Telemetry telemetry;

    protected MechComponent(IControlStrategy strategy)
    {
        this.strategy = strategy;
    }

    public void initializeTelemetry(Telemetry telemetry) {
        this.telemetry = telemetry;
    }

    abstract void move(Gamepad gamepad);
    abstract void update();
}

```

MecanumDrive.java

```

package org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.Mecanum;

import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.hardware.IMU;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.Extensions.AngleExtensions;
import org.firstinspires.ftc.teamcode.Extensions.GamepadExtensions;
import org.firstinspires.ftc.teamcode.Extensions.ThreadExtensions;
import org.firstinspires.ftc.teamcode.Extensions.TurnDirection;
import org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.DriveTrain;

public class MecanumDrive extends DriveTrain
{
    public class AutonomousMecanumDrive extends AutonomousDriving
    {
        public AutonomousMecanumDrive(Telemetry telemetry) {
            super(telemetry);
        }

        public void turnToAngle(double degrees) {
            degrees = AngleExtensions.mapToIMURange(degrees);
            double yaw = imu.getRobotYawPitchRollAngles().getYaw();
            double smol = AngleExtensions.getSmol(degrees, yaw);

            reportData("Target Angle", degrees);
            reportData("Current Yaw", yaw);
            reportData("Error", smol);

            while(Math.abs(smol) > 1 ) {
                if (smol > 0) {
                    spin(TurnDirection.LEFT);
                    reportStatus("Turning left to target");
                }
                else {
                    spin(TurnDirection.RIGHT);
                    reportStatus("Turning right to target");
                }
                ThreadExtensions.TrySleep(100);
                yaw = imu.getRobotYawPitchRollAngles().getYaw();
                smol = AngleExtensions.getSmol(degrees, yaw);
            }
        }
    }
}

```

```

    }
    reportStatus("Turn complete");
}

public void drive(double x, double y, double t)
{
    double angle = Math.atan2(y, x);
    double magnitude = Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));

    double y1 = Math.sin(angle + Math.PI/4) * magnitude;
    double y2 = Math.sin(angle - Math.PI/4) * magnitude;

    double lf = y1 + t;
    double lb = y2 + t;
    double rf = y2 - t;
    double rb = y1 - t;

    double n = Math.max(
        Math.abs(lf),
        Math.max(
            Math.abs(lb),
            Math.max(
                Math.abs(rf),
                Math.abs(rb)))));

    if (n > 1.0)
    {
        lf/=n;
        lb/=n;
        rf/=n;
        rb/=n;
    }

    RF.setPower(rf);
    RB.setPower(rb);
    LF.setPower(lf);
    LB.setPower(lb);

    reportData("Drive X", x);
    reportData("Drive Y", y);
    reportData("Turn", t);
}

public void spin(TurnDirection direction)

```

```

    {
        double dir = direction==TurnDirection.RIGHT? 1:-1;
        LB.setPower(dir);
        LF.setPower(dir);
        RB.setPower(-dir);
        RF.setPower(-dir);
    }
}

```

```

private final DcMotor LB;
private final DcMotor LF;
private final DcMotor RB;
private final DcMotor RF;
private final IMU imu;

```

```

private AutonomousMecanumDrive auton;

```

```

public static class OrientationConfiguration{
    DcMotorSimple.Direction lb, lf, rb, rf;

    public OrientationConfiguration(
        DcMotorSimple.Direction lb,
        DcMotorSimple.Direction lf,
        DcMotorSimple.Direction rb,
        DcMotorSimple.Direction rf
    )
    {
        this.lb = lb;
        this.lf = lf;
        this.rb = rb;
        this.rf = rf;
    }

    public DcMotorSimple.Direction getLb(){
        return lb;
    }

    public DcMotorSimple.Direction getLf(){
        return lf;
    }

    public DcMotorSimple.Direction getRb(){
        return rb;
    }

    public DcMotorSimple.Direction getRf(){
        return rf;
    }
}

```

```
}
```

```
public MecanumDrive(  
    HardwareMap hardwareMap,  
    OrientationConfiguration orientationConfiguration)
```

```
{  
    LB = hardwareMap.get(DcMotor.class, "lb");  
    LB.setDirection(orientationConfiguration.getLb());  
    LF = hardwareMap.get(DcMotor.class, "lf");  
    LF.setDirection(orientationConfiguration.getLf());  
    RB = hardwareMap.get(DcMotor.class, "rb");  
    RB.setDirection(orientationConfiguration.getRb());  
    RF = hardwareMap.get(DcMotor.class, "rf");  
    RF.setDirection(orientationConfiguration.getRf());
```

```
    imu = hardwareMap.get(IMU.class, "imu");  
}
```

```
@Override  
public void initializeTelemetry(Telemetry telemetry) {  
    this.telemetry = telemetry;  
    auton = new AutonomousMecanumDrive(telemetry);  
}
```

```
@Override  
public AutonomousMecanumDrive getAutonomousDriving() {  
    return auton;  
}
```

```
@Override  
public void drive(Gamepad gamepad)  
{  
    double vertical = GamepadExtensions.GetLeftStickY(gamepad);  
    double horizontal = GamepadExtensions.GetLeftStickX(gamepad);  
    double turn = GamepadExtensions.GetRightStickX(gamepad);  
  
    double angle = Math.atan2(vertical, horizontal);  
    double magnitude = Math.sqrt(Math.pow(horizontal, 2) + Math.pow(vertical, 2));  
  
    double y1 = Math.sin(angle + Math.PI/4) * magnitude;  
    double y2 = Math.sin(angle - Math.PI/4) * magnitude;  
  
    double lf = y1 + turn;  
    double lb = y2 - turn;
```

```
double rf = y2 + turn;
double rb = y1 - turn;
```

```
double n = Math.max(
    Math.abs(lf),
    Math.max(
        Math.abs(lb),
        Math.max(
            Math.abs(rf),
            Math.abs(rb)))));
```

```
if (n > 1.0)
{
    lf/=n;
    lb/=n;
    rf/=n;
    rb/=n;
}
```

```
RF.setPower(rf);
RB.setPower(rb);
LF.setPower(lf);
LB.setPower(lb);
}
```

```
@Override
```

```
public void updateTelemetry()
```

```
{
    if (telemetry != null) {
        double yaw = imu.getRobotYawPitchRollAngles().getYaw();
        telemetry.addData("IMU Yaw", "%.2f degrees", yaw);
        telemetry.addData("LF Power", "%.2f", LF.getPower());
        telemetry.addData("RF Power", "%.2f", RF.getPower());
        telemetry.addData("LB Power", "%.2f", LB.getPower());
        telemetry.addData("RB Power", "%.2f", RB.getPower());
    }
}
}
```

OneStickTank.java

```

package org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.Tank;

import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.util.ElapsedTime;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.Extensions.GamepadExtensions;
import org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.DriveTrain;

public class OneStickTank extends DriveTrain
{
    private class LeroyState extends AutonomousDriving
    {
        ElapsedTime _stateTimer;
        double _chargeTime;
        boolean _isActive;

        public LeroyState(double chargeTime, Telemetry telemetry)
        {
            super(telemetry);
            _stateTimer = new ElapsedTime(ElapsedTime.Resolution.SECONDS);
            _chargeTime = chargeTime;
            _isActive = false;
        }

        public void activate()
        {
            left.setPower(1);
            right.setPower(1);
            _stateTimer.reset();
            _isActive = true;
            reportStatus("LEROY JENKINS! Charging for " + _chargeTime + " seconds");
        }

        public boolean isCompleted()
        {
            if(!_isActive)
                return true;

            double duration = _stateTimer.time();
            reportData("Leroy Charge Time", String.format("%.1f/%.1f sec", duration, _chargeTime));
        }
    }
}

```

```

        if(duration < _chargeTime)
            return false;

        left.setPower(0);
        right.setPower(0);
        _isActive = false;
        reportStatus("Leroy charge complete");
        return true;
    }
}

private final DcMotor left, right;
private LeroyState leroyState;

public OneStickTank(HardwareMap hardwareMap)
{
    left = hardwareMap.get(DcMotor.class, "leftTread");
    right = hardwareMap.get(DcMotor.class, "rightTread");
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    this.telemetry = telemetry;
    leroyState = new LeroyState(10, telemetry);
}

@Override
public LeroyState getAutonomousDriving()
{
    return leroyState;
}

@Override
public void drive(Gamepad gamepad)
{
    if(!leroyState.isCompleted())
        return; // Leroy is in control!

    if(gamepad.left_bumper && gamepad.right_bumper)
    {
        leroyState.activate();
        return;
    }
}

```

```

float throttle = GamepadExtensions.GetLeftStickY(gamepad);
float turnBias = GamepadExtensions.GetLeftStickX(gamepad);
float extraTurnBias = GamepadExtensions.GetRightStickX(gamepad);

if( (extraTurnBias < 0 && turnBias < 0) ||
    (extraTurnBias > 0 && turnBias > 0) )
{
    turnBias += extraTurnBias;
    if(Math.abs(turnBias) > 1)
        throttle = Math.max(Math.abs(throttle), Math.abs(turnBias) - 1);
}

left.setPower(calculateLeftPower(throttle, turnBias));
right.setPower(calculateRightPower(throttle, turnBias));
}

private float calculateRightPower(float throttle, float turnBias)
{
    if(turnBias <= 0)
        return throttle;
    return throttle * (1 - turnBias);
}

private float calculateLeftPower(float throttle, float turnBias)
{
    if(turnBias >= 0)
        return throttle;
    return throttle * (1 + turnBias);
}

@Override
public void updateTelemetry()
{
    if (telemetry != null) {
        telemetry.addData("Left Power", "%.2f", left.getPower());
        telemetry.addData("Right Power", "%.2f", right.getPower());
        telemetry.addData("Leroy Active", !leroyState.isCompleted());
    }
}
}

```

SpinnyIntake.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public class SpinnyIntake extends MechComponent
{
    public class AutonomousIntakeBehaviors extends AutonomousComponentBehaviors
    {
        public AutonomousIntakeBehaviors(Telemetry telemetry) {
            super(telemetry);
        }

        public void startIntake()
        {
            intake.setPower(-1);
            reportStatus("Intake started");
            reportData("Intake Power", -1.0);
        }

        public void stopIntake()
        {
            intake.setPower(0);
            reportStatus("Intake stopped");
            reportData("Intake Power", 0.0);
        }

        public void reverseIntake()
        {
            intake.setPower(1);
            reportStatus("Intake reversed");
            reportData("Intake Power", 1.0);
        }
    }

    public interface SpinnyIntakeControlStrategy extends IControlStrategy
    {
        void nomNomNom(DcMotor motor, Gamepad gamepad);
    }

    private final DcMotor intake;

```

```

protected SpinnyIntakeControlStrategy strategy;
private AutonomousIntakeBehaviors auton;

public SpinnyIntake(
    HardwareMap hardwareMap,
    String motorName,
    SpinnyIntakeControlStrategy strategy)
{
    super(strategy);
    intake = hardwareMap.get(DcMotor.class, motorName);
    this.strategy = strategy;
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    super.initializeTelemetry(telemetry);
    auton = new AutonomousIntakeBehaviors(telemetry);
}

@Override
public AutonomousIntakeBehaviors getAutonomousBehaviors() {
    return auton;
}

@Override
public void move(Gamepad gamepad)
{
    strategy.nomNomNom(intake, gamepad);
}

@Override
public void update() {
    if (telemetry != null) {
        telemetry.addData("Intake Power", "%.2f", intake.getPower());
        telemetry.addData("Intake Position", intake.getCurrentPosition());
    }
}
}

```

DoubleShooter.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public class DoubleShooter extends MechComponent
{
    public class AutonomousShooterBehavior extends AutonomousComponentBehaviors {

        public AutonomousShooterBehavior(Telemetry telemetry) {
            super(telemetry);
        }

        public void StartShoot() {
            shooterF.setPower(0.76);
            shooterB.setPower(-0.76);
            reportStatus("Shooter started");
            reportData("Shooter Power", 0.76);
        }

        public void StopShoot(){
            shooterF.setPower(0);
            shooterB.setPower(0);
            reportStatus("Shooter stopped");
            reportData("Shooter Power", 0.0);
        }

        public void setPower(double power){
            shooterF.setPower(power);
            shooterB.setPower(-power);
            reportData("Shooter Power", power);
        }
    }

    public interface ShooterControlStrategy extends IControlStrategy
    {
        void shoot(DcMotor motorF, DcMotor motorB, Gamepad gamepad);
    }

    public interface ShooterTelemetryStrategy
    {

```

```
    void update(DcMotor motorF, DcMotor motorB, Telemetry telemetry);  
}
```

```
private final DcMotor shooterF;  
private final DcMotor shooterB;  
protected ShooterControlStrategy pewpew;  
protected ShooterTelemetryStrategy telemetryStrategy;  
private AutonomousShooterBehavior auton;
```

```
public DoubleShooter(  
    HardwareMap hardwareMap,  
    String motorNameF, String motorNameB,  
    ShooterControlStrategy pewpew,  
    ShooterTelemetryStrategy telemetryStrategy) {  
    super(pewpew);  
    shooterF = hardwareMap.get(DcMotor.class, motorNameF);  
    shooterB = hardwareMap.get(DcMotor.class, motorNameB);  
    this.pewpew = pewpew;  
    this.telemetryStrategy = telemetryStrategy;  
}
```

```
@Override  
public void initializeTelemetry(Telemetry telemetry) {  
    super.initializeTelemetry(telemetry);  
    auton = new AutonomousShooterBehavior(telemetry);  
}
```

```
@Override  
public AutonomousShooterBehavior getAutonomousBehaviors() {  
    return auton;  
}
```

```
@Override  
public void move(Gamepad gamepad) {  
    pewpew.shoot(shooterF, shooterB, gamepad);  
}
```

```
@Override  
public void update() {  
    if (telemetry != null && telemetryStrategy != null) {  
        telemetryStrategy.update(shooterF, shooterB, telemetry);  
    } else if (telemetry != null) {  
        telemetry.addData("Shooter F Power", "%.2f", shooterF.getPower());  
        telemetry.addData("Shooter B Power", "%.2f", shooterB.getPower());  
    }  
}
```

}
}
}

PusherServo.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.hardware.Servo;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public class PusherServo extends MechComponent {

    public class AutonomousBallPusherBehaviors extends AutonomousComponentBehaviors {

        public AutonomousBallPusherBehaviors(Telemetry telemetry) {
            super(telemetry);
        }

        public void pushBalls() {
            servoR.setPosition(0.8);
            reportStatus("Pushing balls");
            reportData("Pusher Position", 0.8);
        }

        public void setPosition(double position) {
            servoR.setDirection(Servo.Direction.REVERSE);
            servoR.setPosition(position);
            reportData("Pusher Position", position);
        }

        public void retractPusher() {
            servoR.setPosition(0.0);
            reportStatus("Pusher retracted");
            reportData("Pusher Position", 0.0);
        }
    }

    public interface BallPusherControlStrategy extends IControlStrategy {
        void controlPusher(Servo servoR, Gamepad gamepad);
    }

    public interface PusherTelemetryStrategy
    {
        void update(Servo servo, Telemetry telemetry);
    }

```

```

private final Servo servoR;
private final BallPusherControlStrategy strategy;
protected final PusherTelemetryStrategy telemetryStrategy;
private AutonomousBallPusherBehaviors auton;

public PusherServo(HardwareMap hardwareMap,
                  String servoNameR,
                  BallPusherControlStrategy strategy,
                  PusherTelemetryStrategy telemetryStrategy) {
    super(strategy);
    this.servoR = hardwareMap.get(Servo.class, servoNameR);
    this.strategy = strategy;
    this.telemetryStrategy = telemetryStrategy;
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    super.initializeTelemetry(telemetry);
    auton = new AutonomousBallPusherBehaviors(telemetry);
}

@Override
public AutonomousBallPusherBehaviors getAutonomousBehaviors() {
    return auton;
}

@Override
public void move(Gamepad gamepad) {
    strategy.controlPusher(servoR, gamepad);
}

@Override
public void update() {
    if (telemetry != null && telemetryStrategy != null) {
        telemetryStrategy.update(servoR, telemetry);
    } else if (telemetry != null) {
        telemetry.addData("Right Pusher Position", "%.2f", servoR.getPosition());
    }
}
}

```

Turret.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.CRServo;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.Extensions.ThreadExtensions;

public class Turret extends MechComponent
{
    public class AutonomousTurretBehaviors extends AutonomousComponentBehaviors
    {
        public AutonomousTurretBehaviors(Telemetry telemetry) {
            super(telemetry);
        }

        public void setPower(double power)
        {
            servo.setPower(power);
            reportData("Turret Power", power);
        }

        public void turnCCW() {
            servo.setPower(1);
            reportStatus("Turning turret CCW");
            ThreadExtensions.TrySleep(100);
        }

        public void turnCW() {
            servo.setPower(-1);
            reportStatus("Turning turret CW");
            ThreadExtensions.TrySleep(100);
        }

        public void stop() {
            servo.setPower(0);
            reportStatus("Turret stopped");
        }
    }

    public interface TurretControlStrategy extends IControlStrategy
    {
        void move(CRServo servo, Gamepad gamepad);
    }
}

```

```

}

public interface TurretTelemetryStrategy
{
    void update(CRServo servo, Telemetry telemetry);
}

public CRServo servo;
protected TurretControlStrategy strategy;
protected TurretTelemetryStrategy telemetryStrategy;
private AutonomousTurretBehaviors auton;

public Turret(HardwareMap hardwareMap,
              String servoName,
              TurretControlStrategy strategy,
              TurretTelemetryStrategy telemetryStrategy)
{
    super(strategy);
    servo = hardwareMap.get(CRServo.class, servoName);
    this.strategy = strategy;
    this.telemetryStrategy = telemetryStrategy;
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    super.initializeTelemetry(telemetry);
    auton = new AutonomousTurretBehaviors(telemetry);
}

@Override
public AutonomousTurretBehaviors getAutonomousBehaviors()
{
    return auton;
}

@Override
public void move(Gamepad gamepad)
{
    strategy.move(servo, gamepad);
}

@Override
public void update()
{

```

```
    if (telemetry != null && telemetryStrategy != null) {  
        telemetryStrategy.update(servo, telemetry);  
    } else if (telemetry != null) {  
        telemetry.addData("Turret Power", "%.2f", servo.getPower());  
    }  
}  
}
```

Claw.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.CRServo;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public class Claw extends MechComponent
{
    public class AutonomousClawBehaviors extends AutonomousComponentBehaviors
    {
        public AutonomousClawBehaviors(Telemetry telemetry) {
            super(telemetry);
        }

        public void open()
        {
            servo.setPower(1);
            reportStatus("Claw opening");
            reportData("Claw Power", 1.0);
        }

        public void close()
        {
            servo.setPower(-1);
            reportStatus("Claw closing");
            reportData("Claw Power", -1.0);
        }

        public void stop()
        {
            servo.setPower(0);
            reportStatus("Claw stopped");
            reportData("Claw Power", 0.0);
        }
    }

    public interface ClawControlStrategy extends IControlStrategy
    {
        void chomp(CRServo servo, Gamepad gamepad);
    }

    public interface ClawTelemetryStrategy

```

```

{
    void update(CRServo servo, Telemetry telemetry);
}

public CRServo servo;
protected ClawControlStrategy strategy;
protected ClawTelemetryStrategy telemetryStrategy;
private AutonomousClawBehaviors auton;

public Claw(HardwareMap hardwareMap,
            String servoName,
            ClawControlStrategy strategy,
            ClawTelemetryStrategy telemetryStrategy)
{
    super(strategy);
    servo = hardwareMap.get(CRServo.class, servoName);
    this.strategy = strategy;
    this.telemetryStrategy = telemetryStrategy;
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    super.initializeTelemetry(telemetry);
    auton = new AutonomousClawBehaviors(telemetry);
}

@Override
public AutonomousClawBehaviors getAutonomousBehaviors()
{
    return auton;
}

@Override
public void move(Gamepad gamepad)
{
    strategy.chomp(servo, gamepad);
}

@Override
public void update()
{
    if (telemetry != null && telemetryStrategy != null) {
        telemetryStrategy.update(servo, telemetry);
    } else if (telemetry != null) {

```

```
        telemetry.addData("Claw Power", "%.2f", servo.getPower());  
    }  
}  
}
```

DoublyLimitedMotor.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components;

import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.hardware.TouchSensor;

import org.firstinspires.ftc.robotcore.external.Telemetry;

public class DoublyLimitedMotor extends MechComponent{

    public class AutonomousDLMBehaviors extends AutonomousComponentBehaviors
    {
        public AutonomousDLMBehaviors(Telemetry telemetry) {
            super(telemetry);
        }

        public void goForward()
        {
            setPower(1);
            reportStatus("Motor going forward");
        }

        public void stop()
        {
            setPower(0);
            reportStatus("Motor stopped");
        }

        public void goBackward()
        {
            setPower(-1);
            reportStatus("Motor going backward");
        }
    }

    public interface DoublyLimitedMotorControlStrategy extends IControlStrategy{
        void move(Gamepad gamepad, DoublyLimitedMotor doublyLimitedMotor);
    }

    public interface DoublyLimitedMotorTelemetryStrategy {
        void update(DoublyLimitedMotor dlm, Telemetry telemetry);
    }

```

```

private DcMotor motor;
private TouchSensor forwardSensor;
private TouchSensor reverseSensor;
private DoublyLimitedMotorControlStrategy strategy;
protected DoublyLimitedMotorTelemetryStrategy telemetryStrategy;
private AutonomousDLMBehaviors auton;

public DoublyLimitedMotor(
    HardwareMap hardwareMap,
    String motorName,
    String forwardSensorName,
    String reverseSensorName,
    DoublyLimitedMotorControlStrategy strategy,
    DoublyLimitedMotorTelemetryStrategy telemetryStrategy
) {
    super(strategy);
    motor = hardwareMap.get(DcMotor.class, motorName);
    forwardSensor = hardwareMap.get(TouchSensor.class, forwardSensorName);
    reverseSensor = hardwareMap.get(TouchSensor.class, reverseSensorName);
    this.strategy = strategy;
    this.telemetryStrategy = telemetryStrategy;
}

@Override
public void initializeTelemetry(Telemetry telemetry) {
    super.initializeTelemetry(telemetry);
    auton = new AutonomousDLMBehaviors(telemetry);
}

public void setPower(double power){
    if ((power > 0 && !canGoForward()) || (power < 0 && !canGoReverse())) {
        power = 0;
    }
    motor.setPower(power);
}

@Override
public AutonomousDLMBehaviors getAutonomousBehaviors() {
    return auton;
}

@Override
public void move(Gamepad gamepad) {
    strategy.move(gamepad, this);
}

```

```
}

@Override
public void update()
{
    if (telemetry != null && telemetryStrategy != null) {
        telemetryStrategy.update(this, telemetry);
    } else if (telemetry != null) {
        telemetry.addData("Motor Power", "%.2f", motor.getPower());
        telemetry.addData("Can Go Forward", canGoForward());
        telemetry.addData("Can Go Reverse", canGoReverse());
    }
}

public boolean canGoForward()
{
    return !forwardSensor.isPressed();
}

public boolean canGoReverse()
{
    return !reverseSensor.isPressed();
}
}
```

BillyMA.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies;

import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.hardware.Servo;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.AutonStrategies.BillyRapidFire;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.DoubleShooter;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.SpinnyIntake;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.PusherServo;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.Turret;

public class BillyMA extends MechAssembly {

    public class AutonomousBillyMA extends AutonomousMechBehaviors {
        public final SpinnyIntake.AutonomousIntakeBehaviors autonIntake;
        public final PusherServo.AutonomousBallPusherBehaviors autonBallPusher;
        public final DoubleShooter.AutonomousShooterBehavior autonFlywheel;
        public final Turret.AutonomousTurretBehaviors autonTurret;

        public AutonomousBillyMA(
            SpinnyIntake.AutonomousIntakeBehaviors autonIntake,
            PusherServo.AutonomousBallPusherBehaviors autonBallPusher,
            DoubleShooter.AutonomousShooterBehavior autonFlywheel,
            Turret.AutonomousTurretBehaviors autonTurret,
            Telemetry telemetry) {
            super(telemetry);
            this.autonIntake = autonIntake;
            this.autonBallPusher = autonBallPusher;
            this.autonFlywheel = autonFlywheel;
            this.autonTurret = autonTurret;
        }
    }

    private final SpinnyIntake intake;
    private final PusherServo ballPusher;
    private final DoubleShooter flywheel;
    private final Turret turret;
    private BillyRapidFire BRF = null;
    private AutonomousBillyMA auton;

    public BillyMA(HardwareMap hardwareMap) {
        intake = new SpinnyIntake(hardwareMap, "intakeMotor",

```

```

(motor, gamepad) -> {
    if (BRF != null && !BRF.isComplete()) {
        return;
    }
    if (gamepad.dpad_up) {
        motor.setPower(0.75);
    } else if (gamepad.dpad_down) {
        motor.setPower(-0.75);
    } else {
        motor.setPower(0);
    }
}
});

```

```

ballPusher = new PusherServo(hardwareMap,
    "ballPusherServo",
    (servoR, gamepad) -> {
        if (BRF != null && !BRF.isComplete()) {
            return;
        }
        servoR.setDirection(Servo.Direction.REVERSE);
        if (gamepad.x) {
            servoR.setPosition(0.8);
        } else {
            servoR.setPosition(0.0);
        }
    },
    (servo, telemetry) -> {
        telemetry.addData("pusher position", servo.getPosition());
    }
});

```

```

flywheel = new DoubleShooter(hardwareMap, "flywheelMotorF", "flywheelMotorB",
    (motorF, motorB, gamepad) -> {
        if (BRF != null && !BRF.isComplete()) {
            return;
        }
        double power = 0.45;
        if (gamepad.dpad_up) {
            power += 0.05;
        }
        if (gamepad.dpad_down) {
            power -= 0.05;
        }

        if (gamepad.y) {

```

```

        motorF.setPower(power);
        motorB.setPower(-power);
    } else {
        motorF.setPower(0);
        motorB.setPower(0);
    }
},
(motorF, motorB, telemetry) -> {
    telemetry.addData("Flywheel F Power", motorF.getPower());
    telemetry.addData("Flywheel B Power", motorB.getPower());
});

turret = new Turret(hardwareMap, "turretServo",
    (servo, gamepad) -> {
        if (BRF != null && !BRF.isComplete()) {
            return;
        }
        if (gamepad.right_bumper) {
            servo.setPower(-1);
        } else if (gamepad.left_bumper) {
            servo.setPower(1);
        } else {
            servo.setPower(0);
        }
    },
    (servo, telemetry) -> {
        telemetry.addData("turret power", servo.getPower());
    });
}

```

@Override

```

public void initializeTelemetry(Telemetry telemetry) {
    this.telemetry = telemetry;

    // Initialize all components with telemetry
    intake.initializeTelemetry(telemetry);
    ballPusher.initializeTelemetry(telemetry);
    flywheel.initializeTelemetry(telemetry);
    turret.initializeTelemetry(telemetry);

    // Create autonomous behaviors
    auton = new AutonomousBillyMA(
        intake.getAutonomousBehaviors(),
        ballPusher.getAutonomousBehaviors(),

```

```

        flywheel.getAutonomousBehaviors(),
        turret.getAutonomousBehaviors(),
        telemetry
    );

    // Initialize BillyRapidFire with new telemetry approach
    BRF = new BillyRapidFire(auton, 3);
    BRF.abort();
}

@Override
public AutonomousBillyMA getAutonomousBehaviors() {
    return auton;
}

@Override
public void giveInstructions(Gamepad gamepad) {
    intake.move(gamepad);
    ballPusher.move(gamepad);
    flywheel.move(gamepad);
    turret.move(gamepad);

    if (gamepad.a && BRF != null && BRF.isComplete()) {
        BRF.reset(3);
        if (telemetry != null) {
            telemetry.addLine("Start Rapid Fire");
        }
    }
    if (BRF != null && !BRF.isComplete()) {
        BRF.updateState();
    }
}

@Override
public void updateTelemetry() {
    if (telemetry != null) {
        telemetry.addLine("=== Billy Mech Assembly ===");

        // Report all component telemetry
        intake.update();
        ballPusher.update();
        flywheel.update();
        turret.update();
    }
}

```

```
// Report assembly-level status
if (BRF != null) {
    telemetry.addData("Rapid Fire Active", !BRF.isComplete());
}
}
}
```

CascadeArm.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies;

import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.teamcode.Extensions.GamepadExtensions;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.Claw;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.DoublyLimitedMotor;

public class CascadeArm extends MechAssembly
{
    public class AutonomousCascadeArm extends AutonomousMechBehaviors
    {
        public final DoublyLimitedMotor.AutonomousDLMBehaviors cascade;
        public final DoublyLimitedMotor.AutonomousDLMBehaviors drawbridge;
        public final Claw.AutonomousClawBehaviors claw;

        public AutonomousCascadeArm(
            DoublyLimitedMotor.AutonomousDLMBehaviors cascade,
            DoublyLimitedMotor.AutonomousDLMBehaviors drawbridge,
            Claw.AutonomousClawBehaviors claw,
            Telemetry telemetry)
        {
            super(telemetry);
            this.cascade = cascade;
            this.drawbridge = drawbridge;
            this.claw = claw;
        }
    }

    private final DoublyLimitedMotor cascade;
    private final DoublyLimitedMotor drawbridge;
    private final Claw claw;
    private AutonomousCascadeArm auton;

    public CascadeArm(HardwareMap hardwareMap) {
        cascade = new DoublyLimitedMotor(
            hardwareMap,
            "cascadeMotor",
            "topLiftLimit",
            "bottomLiftLimit",
            (gamepad, dlm) -> {
                double power = GamepadExtensions.GetLeftStickY(gamepad);

```

```

        dlm.setPower(power);
    },
    (dlm, telemetry) -> {
        telemetry.addData("Cascade Can Go Up", dlm.canGoForward());
        telemetry.addData("Cascade Can Go Down", dlm.canGoReverse());
    });

drawbridge = new DoublyLimitedMotor(
    hardwareMap,
    "drawbridge",
    "topDrawLimit",
    "botDrawLimit",
    (gamepad, dlm) -> {
        double power = GamepadExtensions.GetRightStickY(gamepad);
        dlm.setPower(power);
    },
    (dlm, telemetry) -> {
        telemetry.addData("Drawbridge Can Go Up", dlm.canGoForward());
        telemetry.addData("Drawbridge Can Go Down", dlm.canGoReverse());
    });

claw = new Claw(
    hardwareMap,
    "servo",
    (servo, gamepad) -> {
        if (gamepad.a) servo.setPower(1.0);
        else if (gamepad.b) servo.setPower(-1.0);
        else servo.setPower(0);
    },
    (servo, telemetry) -> {
        telemetry.addData("Claw Power", servo.getPower());
    });
}

```

```

@Override
public void initializeTelemetry(Telemetry telemetry) {
    this.telemetry = telemetry;

    cascade.initializeTelemetry(telemetry);
    drawbridge.initializeTelemetry(telemetry);
    claw.initializeTelemetry(telemetry);

    auton = new AutonomousCascadeArm(
        cascade.getAutonomousBehaviors(),

```

```

        drawbridge.getAutonomousBehaviors(),
        claw.getAutonomousBehaviors(),
        telemetry);
    }

    @Override
    public AutonomousCascadeArm getAutonomousBehaviors()
    {
        return auton;
    }

    @Override
    public void giveInstructions(Gamepad gamepad) {
        cascade.move(gamepad);
        drawbridge.move(gamepad);
        claw.move(gamepad);
    }

    @Override
    public void updateTelemetry() {
        if (telemetry != null) {
            telemetry.addLine("=== Cascade Arm Assembly ===");
            cascade.update();
            drawbridge.update();
            claw.update();
        }
    }
}

```

BillyRobot.java

```

package org.firstinspires.ftc.teamcode.RobotModel.Robots;

import com.qualcomm.hardware.limelightvision.Limelight3A;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.robotcore.hardware.Gamepad;
import com.qualcomm.robotcore.hardware.HardwareMap;
import com.qualcomm.robotcore.hardware.IMU;
import com.qualcomm.hardware.rev.RevHubOrientationOnRobot;

import org.firstinspires.ftc.robotcore.external.Telemetry;
import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
import org.firstinspires.ftc.robotcore.external.navigation.YawPitchRollAngles;
import org.firstinspires.ftc.teamcode.AutonStrategies.LimelightAutoTarget;
import org.firstinspires.ftc.teamcode.RobotModel.DriveTrain.Mecanum.MecanumDrive;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies.BillyMA;
import org.firstinspires.ftc.vision.apriltag.AprilTagProcessor;

public class BillyRobot extends Robot {
    public class AutonomousMecanumRobot extends AutonomousRobot
    {
        public final MecanumDrive.AutonomousMecanumDrive driveTrain;
        public final BillyMA.AutonomousBillyMA mechAssembly;

        public AutonomousMecanumRobot(MecanumDrive.AutonomousMecanumDrive driveTrain,
                                       BillyMA.AutonomousBillyMA mechAssembly,
                                       Telemetry telemetry)
        {
            super(driveTrain, mechAssembly, telemetry);
            this.driveTrain = driveTrain;
            this.mechAssembly = mechAssembly;
        }

        public double getHeading() {
            YawPitchRollAngles orientation = imu.getRobotYawPitchRollAngles();
            return orientation.getYaw(AngleUnit.DEGREES);
        }

        public double angleWrap(double angle)
        {
            while (angle > 180) angle -= 360;
            while (angle < -180) angle += 360;
            return angle;
        }
    }
}

```

```

private final AprilTagProcessor aprilTag;
private final AutonomousMecanumRobot auton;
public final Limelight3A limelight;
public final IMU imu;
public final LimelightAutoTarget targeter;

public BillyRobot(HardwareMap hardwareMap, Telemetry telemetry, int tagID) {
    super(telemetry);

    driveTrain = new MecanumDrive(hardwareMap, new
MecanumDrive.OrientationConfiguration(
        DcMotorSimple.Direction.FORWARD,
        DcMotorSimple.Direction.FORWARD,
        DcMotorSimple.Direction.REVERSE,
        DcMotorSimple.Direction.FORWARD
    ));

    mechAssembly = new BillyMA(hardwareMap);

    limelight = hardwareMap.get(Limelight3A.class, "limelight");
    limelight.setPollRateHz(100);
    limelight.start();
    limelight.pipelineSwitch(0);

    imu = hardwareMap.get(IMU.class, "imu");
    IMU.Parameters parameters = new IMU.Parameters(
        new RevHubOrientationOnRobot(
            RevHubOrientationOnRobot.LogoFacingDirection.RIGHT,
            RevHubOrientationOnRobot.UsbFacingDirection.UP
        )
    );
    imu.initialize(parameters);
    imu.resetYaw();

    aprilTag = AprilTagProcessor.easyCreateWithDefaults();

    // Initialize all subsystems with telemetry
    initializeSubsystems();

    auton = new AutonomousMecanumRobot(
        driveTrain.getAutonomousDriving(),
        mechAssembly.getAutonomousBehaviors(),
        telemetry);

```

```

        targeter = new LimelightAutoTarget(
            limelight,
            ((BillyMA)mechAssembly).getAutonomousBehaviors().autonTurret,
            tagID);
    }

    @Override
    public BillyRobot.AutonomousMecanumRobot getAutonomousRobot() {
        return auton;
    }

    @Override
    public void update(Gamepad gamepad1, Gamepad gamepad2) {
        super.update(gamepad1, gamepad2);
        if(!targeter.isComplete())
            targeter.updateState();
    }

    @Override
    public void updateTelemetry() {
        if (telemetry != null) {
            telemetry.addLine("=== Billy Robot ===");
            telemetry.addData("Target Tag ID", targeter.getTargetTagId());
            telemetry.addData("Targeter Active", !targeter.isComplete());
        }
        super.updateTelemetry();
    }
}

```

BillyRapidFire.java

```

package org.firstinspires.ftc.teamcode.AutonStrategies;

import org.firstinspires.ftc.teamcode.Extensions.IState;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Assemblies.BillyMA;

public class BillyRapidFire extends StateMachine
{
    private final BillyMA.AutonomousBillyMA mechAssembly;
    private int ballCount;

    public BillyRapidFire(BillyMA.AutonomousBillyMA ma, int ballCount)
    {
        mechAssembly = ma;
        reset(ballCount);
    }

    public void reset(int ballCount)
    {
        this.ballCount = ballCount;
        currentState = startShooter();
    }

    public IState startShooter() {
        mechAssembly.reportStatus("Starting shooter");
        return doAndWait(
            () -> mechAssembly.autonFlywheel.setPower(0.6),
            2200,
            fire()
        );
    }

    public IState fire()
    {
        mechAssembly.reportStatus("Firing ball " + (4 - ballCount));
        return doAndWait(
            mechAssembly.autonBallPusher::pushBalls,
            1000,
            retract()
        );
    }

    public IState retract()
    {
        mechAssembly.reportStatus("Retracting pusher");
    }
}

```

```

return doAndWait(
    mechAssembly.autonBallPusher::retractPusher,
    1000,
    --ballCount > 0 ? fire() : stopShooter()
);
}

public IState stopShooter()
{
    mechAssembly.reportStatus("Stopping shooter");
    return () -> {
        mechAssembly.autonFlywheel.StopShoot();
        mechAssembly.reportStatus("Rapid fire complete");
        return null;
    };
}
}

```

LimelightAutoTarget.java

```

package org.firstinspires.ftc.teamcode.AutonStrategies;

import com.qualcomm.hardware.limelightvision.LLResultTypes;
import com.qualcomm.hardware.limelightvision.Limelight3A;

import org.firstinspires.ftc.teamcode.Extensions.IState;
import org.firstinspires.ftc.teamcode.Extensions.LimelightExtensions;
import org.firstinspires.ftc.teamcode.RobotModel.Mechs.Components.Turret;

public class LimelightAutoTarget extends StateMachine {
    private final int targetTagId;
    private final Limelight3A limelight;
    private final Turret.AutonomousTurretBehaviors turret;

    public LimelightAutoTarget(
        Limelight3A limelight,
        Turret.AutonomousTurretBehaviors turret,
        int tagId)
    {
        this.limelight = limelight;
        this.turret = turret;
        this.targetTagId = tagId;
        currentState = lookForTag();
    }

    public int getTargetTagId() {
        return targetTagId;
    }

    public IState rotateCCW(double tx){
        return () -> {
            double power = 1;
            if(tx > -10)
                power = -tx / 10.0;
            turret.setPower(power);
            turret.reportData("Turret CCW Power", power);
            return lookForTag();
        };
    }

    public IState rotateCW(double tx) {
        return () -> {
            double power = 1;
            if (tx < 10)

```

```

        power = -tx / 10.0;
        turret.setPower(power);
        turret.reportData("Turret CW Power", power);
        return lookForTag();
    };
}

public IState stopTurret() {
    return () -> {
        turret.stop();
        turret.reportStatus("Turret on target");
        return lookForTag();
    };
}

public IState lookForTag() {
    return () -> {
        LLResultTypes.FiducialResult tag =
            LimelightExtensions.tryGetFiducial(limelight, targetTagId);

        if(tag == null) {
            turret.reportStatus("Tag " + targetTagId + " not found");
            return stopTurret();
        }

        double tx = tag.getTargetXDegrees();
        turret.reportData("Tag X Offset", tx);

        if(tx < -2) return rotateCCW(tx);
        else if(tx > 2) return rotateCW(tx);
        else return stopTurret();
    };
}
}

```

Reflection

Design Tradeoffs Made

Constructor Complexity vs Runtime Flexibility

Tradeoff: We chose initialization methods over constructor injection for most layers.

- **Benefit:** Existing robot construction code requires minimal changes
- **Cost:** Two-phase initialization creates potential for incomplete setup
- **Alternative:** Pure constructor injection would be cleaner but require rewriting every robot constructor

Autonomous Telemetry Duplication vs Simplicity

Tradeoff: Every autonomous layer gets identical `reportStatus()` and `reportData()` methods.

- **Benefit:** Consistent API across all autonomous surfaces
- **Cost:** Code duplication and potential method explosion
- **Alternative:** Single telemetry service interface would reduce duplication but add complexity

Single Update Point vs Distributed Control

Tradeoff: Only Robot can call `telemetry.update()`, eliminating lower-layer control.

- **Benefit:** Eliminates duplicate flushes and timing conflicts
- **Cost:** Components can't immediately flush critical messages
- **Alternative:** Coordinated update protocol would be more flexible but much more complex

Key Compromises

Backwards Compatibility Preservation

We kept existing control strategies and telemetry strategies intact rather than redesigning them completely. This maintains working robot behaviors but leaves some architectural inconsistencies in place.

Initialization Timing Dependencies

The two-phase initialization (construct → initialize telemetry) creates ordering dependencies that aren't enforced by the type system. A robot could theoretically run with uninitialized telemetry.

Autonomous Strategy Migration Scope

We migrated the most critical autonomous strategies but left some edge cases (like state machines with complex telemetry needs) partially addressed. Full migration would require touching every autonomous helper class.

Follow-up Work Still Needed

1. Null Safety Hardening

Current implementation has defensive null checks, but a more robust approach would:

- Use Optional types
- Implement a NullTelemetry pattern for testing
- Add compile-time initialization verification

2. Testing Infrastructure

The new architecture needs:

- Mock telemetry implementations for unit testing
- Integration tests verifying single-update behavior
- Autonomous telemetry verification helpers

3. Performance Analysis

We should measure:

- Telemetry update frequency and timing
- Memory overhead of autonomous behavior objects
- Impact on loop cycle times

4. Documentation and Training

Teams need:

- Updated architectural documentation
- Migration guide for custom components
- Best practices for autonomous telemetry usage

5. Remaining Dead Telemetry

Some components still have unused telemetry hooks:

- BallDetectionComponent telemetry integration
- Vision pipeline telemetry reporting
- State machine telemetry standardization

6. Advanced Telemetry Features

Future enhancements could include:

- Telemetry filtering and categorization
- Autonomous telemetry recording/playback
- Performance metrics collection